

```

        INP_Default = 0.0,
        ACP_Center = InCenter,
    })

    InWidth:SetAttrs({
        RCP_Center = InCenter,
        RCP_Angle = InAngle,
    })

    InImage = self:AddInput("Input", "Input", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })

    OutImage = self:AddOutput("Output", "Output", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })

end

```

Output

Methods

Sets an image to be output, into the request stream.

Members

None

Example

```
OutImage:Set(req, out)
```

Process

Process() is where the image processing, control and calculations happen. Images, variables, text and numbers from the UI controls are input into the Process function. There are a number of core functions and processes available to manipulate images and pixels.

Image Processing Function

The engine has a number of Image processing routines builtin, the core set of tools like Blur, Merge, Transform and Color Operators, all optimized for features and performance.

BlendOf

Summary

The BlendOf function will blend one image with another image. A numerical value can be used to specify how much of the foreground is blended with the background, or an image can be used to provide a map providing different blend values for each pixel. This is the function used internally by Fusion when a tools blend slider is adjusted.

The function returns a new image containing the results of the blend operation.

Usage

Image:BlendOf(*image* fg, *numeric* val) OR

Image:BlendOf(*image* fg, *image* map)

fg (*image*, required)

The image to use as the foreground for the blend operation.

val (*numeric*, required)

A numeric value that describes how much of the foreground is combined with the background. Alternatively, this argument can be an image map, as described below.

map (*image*, required)

A map image that describes how the pixels from the foreground should be combined with the background. The value of the pixels in the map image provide the amount of Blend. Map value of zero will be background, and map value 1 will be foreground.

Example

This example blends one image with another using a third image as a map.

```
function Process(req)
  local img_bg = InBG:GetValue(req)
  local img_fg = InFG:GetValue(req)
  local map = InMap:GetValue(req)

  img = img_bg:BlendOf(img_fg, map)

  OutImage:Set(req, img)
end
```

Blur

Summary

The Blur function will blur the image. The function returns a new image containing the results of the blur. Alternatively, if the first argument specifies an already existing image object, then the results will be copied into that image instead. In order to give the user a blur slider that works like the standard blur tool, divide size by 720 before calling the Blur

Usage

Image:Blur(image dest_image, table options)

dest_image (image, optional)

The image object where the results of the blur will be applied. If none is provided, a new image will be created.

options (table, required)

A table containing values which describe the various options available for the blur. See the attributes below.

Options Table

— **BLUR_Type**

The blur type is a string which represents the type of blur applied to the image. Valid options are: "Box", "Soften", "Bartlett", "Sharpen", "Gaussian", "Highlight", "Blend", "Solarise"

— **BLUR_Red, BLUR_Green, BLUR_Blue, BLUR_Alpha**

These four tags are used to tell the Blur function which channels of the image to affect. A value of true enables blur for the channel. A value of false disables it. The default behaviour if this tag is not specified is true.

— **BLUR_XSize, BLUR_YSize**

These tags specify the strength of the blur along the X and Y axis, respectively. A value of 1 represents a blur equal to the width of the image.

— **BLUR_Blend**

A value between 0 and 1 which indicates how much of the original input image to blend into the result of the Blur. Unlike the Blend slider presented by the Common Controls tab, this blend takes place inside the blur itself, and thus may be somewhat faster. If not specified the default value will be 1.

— **BLUR_Normalize**

BLUR_Normalize is used when applying Glow to the blurred image. To match what the Glow tool does pass in "1.0 - glow" as the value for that tag.

— **BLUR_RedScale, BLUR_GreenScale, BLUR_BlueScale, BLUR_AlphaScale**

The blur scale tags are multipliers for the results of the glow applied with BLUR_Normalize. These match the glow tool's "Color Scale" options.

Example

```
function Process(req)
  local img = InImage:GetValue(req)
  local blur_strength = InBlur:GetValue(req).Value
  local result = Image({IMG_Like = img})

  img:Blur(result, {
    BLUR_Type = "Gaussian",
    BLUR_Red = true,
    BLUR_Green = true,
    BLUR_Blue = true,
    BLUR_Alpha = false,
```

```

        BLUR_XSize = blur_strength/img.OriginalWidth,
        BLUR_YSize = blur_strength/img.OriginalWidth,
    })

    OutImage:Set(req, result)
end

```

ChannelOpOf

Summary

The ChannelOpOf function performs operations on image channels from one or more images. It can be used to copy the red channel from one image to the alpha channel of another, or to multiply the RGB color channels of one image by its own alpha. This function essentially provides all the functionality normally found in the Channel Booleans tool.

The function returns a new image which contains the result of its operations.

Usage

Image:ChannelOpOf(*string* operation, *image* fg, *table* options)

operation (*string*, required)

A string that specifies the mathematical operation the function will use when combining channels. Can be one of the following: "Copy", "Add", "Subtract", "Multiply", "Divide", "Max", "Min", "Invert", "Difference", "SignedAdd", "Threshold".

fg (*image*, required)

The image object used as the foreground, or second image in the operation. If this is nil, then the function will use the Image object calling the function as the foreground as well (i.e. "Bg.R" is equivalent to "Bg.R").

options (*table*, required)

A table containing named entries describing how each channel of the background image should be processed to produce the result. Each entry in the table may contain either a numeric constant value, or a string specifying a channel in the foreground or background image, which is combined using the specified operation to produce the desired result. The channels used in the entry are abbreviations, and the values are strings that read either Fg.channel or Bg.channel. Case is not important. See the Options section below for a list of acceptable channel abbreviations. Some example options tables are shown below:

```

{ R = "Fg.R", G = "Fg.G", B = "Fg.B", A = "Fg.A" }
{ R = "bg.a", G = "bg.a", B = "bg.a", A = 1.0 }
{ R = blend, G = blend, B = blend, A = blend }

```

Options Table

Channel Abbreviations (case is not important)

- **R, G, B, A**
Red, Green, Blue and Alpha channels
- **BgR, BgG, BgB, BgA**
The Background Red, Green and Blue channels

- **Z**
Z Buffer Channel
- **Coverage**
Z buffer coverage channel
- **ObjectID, MaterialID**
The ObjectID and MaterialID channels
- **U, V, W**
U,V and W texture map coordinates channels
- **NX, NY, NZ**
XYZ normal channels
- **VectorX, VectorY**
The forward X and Y motion vector channels to the next frame
- **BackVectorX, BackVectorY**
Back X and Y motion vectors to the previous frame
- **DisparityX, DisparityY**
Per pixel Disparity position between 2 images, normally stereo images
- **PositionX, PositionY, PositionZ**
World position channels
- **HLS.H, HLS.L, HLS.S**
Hue, Lightness and Saturation channels
- **YUV.Y, YUV.U, YUV.V**
YUV colorspace channels

Example

This example would copy the RGBA channels in the foreground image to the background and return the results to img_out.

```
img_out = img:ChannelOpOf("Copy", img_fg, { R = "Fg.R", G = "Fg.G", B = "Fg.B", A = "Fg.A" } )
```

This example would subtract foreground's RGBA channels from the background and return the results to img_out.

```
img_out = img:ChannelOpOf("Subtract", img_fg, { R = "Fg.R", G = "Fg.G", B = "Fg.B", A = "Fg.A" } )
```

This example would Add the R, G and B of image_bg to the R, G and B of image_fg, returning the results to img_out.

```
img_out = img_bg:ChannelOpOf("Add", img_fg, { R = "Fg.R", G = "Fg.G", B = "Fg.B", A = "Fg.A" })
```

This example would multiply every pixel in img_bg by the value of blend (in this case 0.5) and return the results to img_out.

```
blend = 0.5
```

```
img_out = img_bg:ChannelOpOf("Multiply", nil, { R = blend, G = blend, B = blend, A = blend })
```

```
This example would clip every pixel in img_bg outside the range of 0.2 .. 0.8, scale the remaining pixels to the range of 0 .. 1 and return the results to img_out.
```

```
blend = 0.5
```

```
out = bg:ChannelOpOf("Threshold", nil, { R = "bg.r", G = "bg.g",  
B = "bg.b", A = "bg.a" }, 0.2, 0.8)
```

```
This will copy the alpha from the foreground image to the background  
output image
```

```
img = img:ChannelOpOf("Copy", img_fg, {A = fg.A})
```

```
This will not affect the RGB channels and on multiply alpha
```

```
img = img:ChannelOpOf("Multiply", nil, {R = nil, G = nil, B = nil,  
A = gain})
```

CopyOf

Summary

The CopyOf method returns a new Image object which is a copy of the current image.

Also Copy() is a generic function that all parameter types may implement. CopyOf() is specific to Image. The only real difference in behaviour is that CopyOf() may be interrupted if the tool is told to abort. Copy() will always complete the copying across of pixel data. It is generally a better idea to use CopyOf() with images, but either will work

Usage

Image:CopyOf()

Example

```
function Process(req)  
    local img = InImage:GetValue(req)  
  
    local out = img:CopyOf()  
  
    OutImage:Set(req, out)  
end
```

CSConvert

Summary

The CSConvert method will convert the image to the specified color space.

Usage

Image:CSConvert(*string* from, *string* to)

from (*string*, required)

A string specifying the colorspace the image is currently in. Can be one of the following values.

"RGB", "HLS", "YUV", "YIQ", "CMY", "HSV", "XYZ", "LAB".

to (*string*, required)

A string specifying the colorspace to convert the image into. Can be one of the following values. "RGB", "HLS", "YUV", "YIQ", "CMY", "HSV", "XYZ", "LAB".

Example

A simple Fuse to convert an Image from RGB to HLS and back again.

```
function Process(req)
    local op = InOperation:GetValue(req).Value
    local img = InImage:GetValue(req)

    local newimg = img:Copy()

    if op == 0 then
        newimg:CSConvert("RGB", "HLS")
    else
        newimg:CSConvert("HLS", "RGB")
    end

    OutImage:Set(req, newimg)
end
```

ErodeDilate

Summary

The ErodeDilate function will apply an erode or dilate process to the image. The function returns a new image containing the results of the operation. Alternatively, if the first argument specifies an already existing image object, then the results will be copied into that image instead.

Usage

Image:ErodeDilate(*image* dest_image, *table* options)

dest_image (*image*, optional)

The image object where the results of the erode/dilate will be applied. If none is provided, a new image will be created.

options (*table*, required)

A table containing values which describe the various options available for the erode/dilate. See the attributes below.

Options Table

— ErDI_AmountX, ErDI_AmountY

These tags specify the strength of the operation along the X and Y axis, respectively. Negative values will produce an erode operation, and positive values will produce a dilate operation. A value of 1 represents an operation equal to the width of the image.

— ErDI_Filter

The filter is a string which represents the 'shape' of the effect. Valid options are: "Box", "Linear", "Gaussian", "Circle"

The circle type requires that `ErDl_AmountX` and `ErDl_AmountY` are either both positive or both negative.

— **ErDl_Red, ErDl_Green, ErDl_Blue, ErDl_Alpha**

These four tags are used to indicate which channels of the image to affect. A value of true enables the operation for the channel. A value of false disables it. The default behaviour if this tag is not specified is true.

Example

```
function Process(req)
    local img = InImage:GetValue(req)
    local amount = InAmount:GetValue(req).Value
    local result = Image({IMG_Like = img})

    img:ErodeDilate(result, {
        ErDl_Filter = "Box",
        ErDl_Red = true,
        ErDl_Green = true,
        ErDl_Blue = true,
        ErDl_Alpha = true,
        ErDl_AmountX = amount/img.OriginalWidth,
        ErDl_AmountY = amount/img.OriginalWidth,
    })

    OutImage:Set(req, result)
end
```

Fill

Summary

The Fill method will fill the image with the color specified by the Pixel object provided as its sole argument.

Fill() will also change the canvas color of an image. If you just want to fill the image (inside its DoD) and preserve the canvas color, you need to save it first using `GetCanvasColor()` and restore it afterwards using `SetCanvasColor()`.

Usage

`img:Fill(object pixel)`

object (pixel, required)

This argument should be set to a pixel object.

Example

```
function Process(req)
    img = InImage:GetValue(req)

    p = Pixel({R = 0.5, G = 0.2, B = 0, A = 1})
```



```

        out = Image({IMG_Like = img})
        out:Fill(p)

        OutImage:Set(req, out)
    end

```

Gamma

Summary

The Gamma method applies the gamma adjustment specified in the method's sole argument to every pixel in the Image. The result is applied directly to the Image object which calls the function. This function does not return a value.

Usage

Image:Gamma(*number r, number g, number b, number a*)

number r, g, b, a (*number, required*)

The amount by which to gamma the image.

Example

```

function Process(req)
    local img = InImage:GetValue(req)
    local gamma = 0.5

    local newimg = img:Copy()

    newimg:Gamma(gamma, gamma, gamma, 1)

    OutImage:Set(req, newimg)
end

```

Gain

Summary

The Gain function multiplies every pixel in the image by the specified value. The result is applied directly to the Image object which calls the function. This function does not return a value.

Usage

Image:Gain(*number r, number g, number b, number a*)

number r, g, b, a (*number, required*)

The amount by which to gain the image.

Example

```

function Process(req)
    local img      = InImage:GetValue(req)
    local gain     = InGain:GetValue(req).Value

```

```

    local newimg = img:Copy()

    newimg:Gain(gain, gain, gain, gain)

    OutImage:Set(req, newimg)
end

```

GetCanvasColor

Summary

The GetCanvasColor function is used to retrieve the canvas color values of an Image.

The canvas color is the "default" pixel color, and is used for any part of the image which is not explicitly defined by pixels. This is usually black/transparent, but can be different after certain operations, such as inverting the image.

Usage

GetCanvasColor(*object* pixel)

Pixel

The Pixel object that will receive the color values of the image's canvas.

Example

```

local p = Pixel()

img:GetCanvasColor(p)

if p.R == 0 and p.G == 0 and p.B == 0 and p.A == 0 then
    print("Image canvas is black/transparent")
else
    print("Image canvas is non-black.")
end

```

GetPixel

Summary

The GetPixel function is used to retrieve the values of a specific pixel in an Image. This uses actual pixel co-ordinates, and must always be within image bounds.

Usage

Image:GetPixel(*integer* x_position, *integer* y_position, *object* pixel)

X_position

The position of the pixel to get on the x axis

Y_position

The position of the pixel to get on the y axis

Pixel

The Pixel object that will receive the color values of the image's pixel.

Example

A simple 8 bit histogram function

```
local p = Pixel()
local histoR = {}
local histoG = {}
local histoB = {}
local histoA = {}
local r,g,b,a

-- initialise the histogram table
for i = 0,255 do
    histoR[i] = 0
    histoG[i] = 0
    histoB[i] = 0
    histoA[i] = 0
end

for y=0,Height-1 do
    if self.Status ~= "OK" then break end

    for x=0,Width-1 do
        img:GetPixel(x,y, p)

        -- convert float 0..1 values into int 0.255
        r = math.floor(p.R * 255)
        g = math.floor(p.G * 255)
        b = math.floor(p.B * 255)
        a = math.floor(p.A * 255)

        -- check for out-of-range colors
        if r >= 0 and r <= 255 then histoR[r] = histoR[r] + 1 end
        if g >= 0 and g <= 255 then histoG[g] = histoG[g] + 1 end
        if b >= 0 and b <= 255 then histoB[b] = histoB[b] + 1 end
        if a >= 0 and a <= 255 then histoA[a] = histoA[a] + 1 end
    end
end
```

Image

Summary

The Image function is used when a new image needs to be created in memory. Its sole argument is a table of attributes which describe the new image. The image function returns a handle to the new Image object.

Usage

Image(*table* image_attributes)

image_attributes

The Image function's only argument is a table of attributes which describe the images width, height, color depth, and so forth. In the majority of cases the width, height, aspect depth and other attributes of the new image should exactly match those of another image already in memory. In that case the attribute table would be as simple as

```
local out = Image({IMG_Like=image})
```

There are occasionally times when it will be necessary to specify one or more of the values explicitly, instead of taking them from another image input. For example, if we wanted to create an image exactly like the image `src_image`, but with a different color depth, we might use :

```
local out = Image({IMG_Like = img, IMG_Depth = src_depth})
```

In a source tool like the native Background tool or the Plasma tool, it is usually necessary to specify all of the Image attributes.

Image Attributes

The following attributes can be used when creating a new Image using the Constructor function.

Name	Description
IMG_Like	Set the IMG_Like attribute to an already existing image to copy that image's attributes to the new image.
IMG_CopyChannels	Set this to false to create an Image with different channels than the IMG_Like Image. Use IMG_Channel to then specify the channels the new Image should contain. If no channels are specified, a 4 channel (RGBA) Image will be created.
IMG_CopyChannelsAux	If set to false, allows you to use IMG_Like but discard the aux channel configuration.
IMG_Height	Set the IMG_Height attribute to an integer value representing the actual height of the image in pixels.
IMG_XScale	Set the IMG_XScale to a numeric value representing the X aspect of the image. For an NTSC D1 format image the value would be 0.9, for example.
IMG_YScale	Set the IMG_YScale to a numeric value representing the Y aspect of the image. For an NTSC D1 format image the value would be 1.0, for example.

Name	Description
IMAT_OriginalWidth	Set the IMAT_OriginalWidth to the original width of the image in pixels. When a composition is in Proxy mode, it is possible that IMG_Width and IMG_Height will differ from the IMAT_OriginalWidth and IMAT_OriginalHeight values.
IMAT_OriginalHeight	Set the IMAT_OriginalHeight to the original height of the image in pixels. When a composition is in Proxy mode, it is possible that IMG_Width and IMG_Height will differ from the IMAT_OriginalWidth and IMAT_OriginalHeight values.
IMG_Depth	Set the IMG_Depth attribute to match the image depth desired for the image. This will be an integer value, using the following table : 1 - Single channel image, 8 integer bits per channel. 2 - Single channel image, 16 integer bits per channel. 3 - Single channel image, 16 float bits per channel. 4 - Single channel image, 32 float bits per channel. 5 - Four channel image, 8 integer bits per channel. 6 - Four channel image, 16 integer bits per channel. 7 - Four channel image, 16 float bits per channel. 8 - Four channel image, 32 float bits per channel.
IMG_Quality	The IMG_Quality attribute is a boolean value which specifies whether the image is High Quality (true) or Interactive Quality (false). The IMG_Quality will be true during a final render, or if the HiQ button in the Time Ruler of the composition is selected. It is usually set by calling the Request:IsQuick function. If you create a temporary image, make sure not to omit IMG_Quality. It defaults to "false" which means that scaling and merging it over another image will use nearest-neighbor filtering (resulting in jagged edges). Either set it according to what the Request object returns or set it to true to always use smooth filtering even when the comp is in low quality mode.
IMG_ProxyScale	The IMG_ProxyScale is an integer value representing the current Proxy scale of the image. For example if the current proxy is 2/1, then this should be set to 2.
IMG_MotionBlurQuality	The IMG_MotionBlurQuality attribute is a boolean value which specifies whether Motion Blur is currently enabled for this image. It is usually set by calling the Request:IsNoMotionBlur function.

Name	Description
IMG_Channel	Specify the channels which should be included in the image using the IMG_Channel table values. This is different from all the ones table values above. It should be specified as shown here. <pre> local imgattr = { IMG_Document = self.Comp, { IMG_Channel = "Red", }, { IMG_Channel = "Green", }, { IMG_Channel = "Blue", }, { IMG_Channel = "Alpha", }, IMG_Width = 1920, IMG_Height = 1080, IMG_XScale = 1.0, IMG_YScale = 1.0, IMG_Quality = not req:IsQuick(), IMG_MotionBlurQuality = not req:IsNoMotionBlur(), } </pre> Valid channel names include : — "Red", "Green", "Blue", "Alpha", — "BgRed", "BgGreen", "BgBlue", "BgAlpha" — "Z", "Coverage" — "Object", "Material" — "U", "V", "W" — "NormalX", "NormalY", "NormalZ" — "PositionX", "PositionY", "PositionZ" — "VectorX", "VectorY" — "BackVectorX", "BackVectorY" — "DisparityX", "DisparityY"
 IMG_DataWindow  IMG_ValidWindow	This is used to define the DoD of an image. DataWindow contains the actual size of the pixel area that may be accessed. Writing outside of its bounds may crash ValidWindow specifies the area for which the image is valid (usually the Region of interest. If a new request that is just a subarea of the old one is performed, no re-rendering will take place. Both attributes are read-only. To set them during image creation, use the IMG_DataWindow and IMG_ValidWindow attributes. Both expect FuRectInt objects which are basically four integer pixel values for the left, bottom, and right, top edges of a rectangle
IMG_NoData	Has to be set to true during a precalc request and will create an image object with all its attributes but without allocating memory for the pixel data. If you assign it the result of request:IsPreCalc() this will work for both precalc and process requests.

Example

The following example Fuse takes two image inputs, creates a new image with exactly the same attributes as the first Image input, then adds the two together.

```
function Process(req)
    local img1 = InImage1:GetValue(req)
    local img2 = InImage2:GetValue(req)

    local out = nil -- fail if we don't meet below conditions

    -- Must have a valid operation function, and images must be same
    dimensions
    if (img1.Width == img2.Width) and (img1.Height == img2.Height) then

        out = Image({IMG_Like = img1})

        out:ProcessPixels(0,0, img1.Width, img1.Height, img1, img2,
func)
        end
        OutImage:Set(req, out)
    end
end
```

The following example creates an image from scratch using the full range of attributes for an Image object.

```
function Process(req)
    local realwidth = Width;
    local realheight = Height;

    -- We'll handle proxy ourselves
    Width = Width / Scale
    Height = Height / Scale
    Scale = 1

    local imgattrs = {
        IMG_Document = self.Comp,
        { IMG_Channel = "Red", },
        { IMG_Channel = "Green", },
        { IMG_Channel = "Blue", },
        { IMG_Channel = "Alpha", },
        IMG_Width = Width,
        IMG_Height = Height,
        IMG_XScale = XAspect,
        IMG_YScale = YAspect,
        IMAT_OriginalWidth = realwidth,
        IMAT_OriginalHeight = realheight,
        IMG_Quality = not req:IsQuick(),
        IMG_MotionBlurQuality = not req:IsNoMotionBlur(),
    }
end
```

```

    if not req:IsStampOnly() then
        imgattrs.IMG_ProxyScale = 1
    end

    if SourceDepth ~= 0 then
        imgattrs.IMG_Depth = SourceDepth
    end

    local img = Image(imgattrs) --Image Creation based on Attributes

    local random = math.random
    local p = Pixel({A=1})

    for y=0,Height-1 do
        if self.Status ~= "OK" then break end

        for x=0,Width-1 do
            p.R = random()
            p.G = random()
            p.B = random()
            img:SetPixel(x,y, p)
        end
    end

    OutImage:Set(req, img)
end

```

Tips for Image Constructor

An image created by using {IMG_Like = ...} is not cleared. Its pixels may contain old data from the image cache. If you don't use the image as a target for methods that overwrite the pixels with valid ones you have to clear it yourself using the Fill method.

If you want to work on a 32bit floating point version of the input image internally to prevent quality loss when chaining multiple color corrections, use the Crop method to copy the original image to a temporary one:

```

local orig = InImage:GetValue(req)
-- copy to a float32 image
local temp32 = Image({IMG_Like = orig, IMG_Depth = 8})
orig:Crop(temp32, {CROP_XOffset = 0, CROP_YOffset = 0})

--- do processing ---

-- copy to an image of the original bit depth
out = Image({IMG_Like = orig})
temp32:Crop(out, {CROP_XOffset = 0, CROP_YOffset = 0})
OutImage:Set(req, out)

```

Single-channel images (IMG_Depth < 5, see Attributes) have an alpha channel only. Methods like Gain() still need to be called with four parameters, the first three will be ignored.

Merge

Summary

The Merge method will merge a FG image over the Image calling the method. The foreground image can also be offset, scaled and rotated. This function supports all of the apply modes and operations supported by the Merge tool.

Usage

Image:Merge(*image* fg, table attributes)

fg (*image*, required)

The Image to be used as the foreground of the merge.

attributes (*table*, required)

A table containing entries which describe how the foreground will be merged over the background. All values are optional, and an empty table will perform a default Additive merge of the foreground over the background. Valid entries include:

MO_EdgeMode	"Black" "Canvas" "Wrap" "Duplicate"
MO_ApplyMode	"Normal" "Merge" "Screen" "Dissolve" "Darken" "Multiply" "ColorBurn" "LinearBurn" "Darker Color" "Lighten" "ColorDodge" "LinearDodge" "LighterColor" "Overlay" "SoftLight" "HardLight" "VividColor" "LinearLight" "PinLight" "Difference" "Exclusion" "Hue" "Saturation" "Color" "Luminosity" "Hypotenuse" "Geometric"
MO_ApplyOperator	"Over" "In" "HeldOut" "Atop" "XOr" "Conjoint" "Disjoint" "Mask" "Stencil" "Under"
MO_DoZ	–
MO_UseOpenGL	–
MO_MustDoCopy	–
MO_BBoxOnly	–
MO_FgZOffset	–
MO_BgZOffset	–
MO_XOffset	–
MO_YOffset	–
MO_XAxis	–
MO_YAxis	–
MO_XSize	–
MO_YSize	–
MO_Angle	–
MO_FgAddSub	–
MO_BgAddSub	–

MO_BurnIn	-
MO_FgRedGain	-
MO_FgGreenGain	-
MO_FgBlueGain	-
MO_FgAlphaGain	-
MO_BgAlphaGain	-

Example

This simple example additively merges the FG over the BG and provides no further options.

```
FuRegisterClass("SimpleMerge", CT_Tool, {
    REGS_Category = "Fuses\\Samples",
    REGS_OpIconString = "SMrg",
    REGS_OpDescription = "Simple Merge",
})

function Create()

    InBackground = self:AddInput("Background", "Background", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })

    InForeground = self:AddInput("Foreground", "Foreground", {
        LINKID_DataType = "Image",
        LINK_Main = 2,
    })

    OutImage = self:AddOutput("Output", "Output", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })

end

function Process(req)
    local bg = InBackground:GetValue(req)
    local fg = InForeground:GetValue(req)

    local out = bg:Copy()
    out:Merge(fg, {MO_ApplyMode = "Merge"})

    OutImage:Set(req, out)
end
```

MergeOf

Summary

The MergeOf method will merge a FG image over the Image calling the method, and returns a new Image containing the result. This is different from Image:Merge in that a new image is produced. The foreground image can also be offset, scaled and rotated. This function supports all of the apply modes and operations supported by the Merge tool.

Usage

image new_image = Image:MergeOf(*image* fg, *table* attributes)

fg (*image*, required)

The Image to be used as the foreground of the merge.

attributes (*table*, required)

A table containing entries which describe how the foreground will be merged over the background. All values are optional, and an empty table will perform a default Additive merge of the foreground over the background. Valid entries include:

MO_EdgeMode	"Black" "Canvas" "Wrap" "Duplicate"
MO_ApplyMode	"Normal" "Merge" "Screen" "Dissolve" "Darken" "Multiply" "ColorBurn" "LinearBurn" "Darker Color" "Lighten" "ColorDodge" "LinearDodge" "LighterColor" "Overlay" "SoftLight" "HardLight" "VividColor" "LinearLight" "PinLight" "Difference" "Exclusion" "Hue" "Saturation" "Color" "Luminosity" "Hypotenuse" "Geometric"
MO_ApplyOperator	"Over" "In" "HeldOut" "Atop" "XOr" "Conjoint" "Disjoint" "Mask" "Stencil" "Under"
MO_DoZ	–
MO_UseOpenGL	–
MO_MustDoCopy	–
MO_BBoxOnly	–
MO_FgZOffset	–
MO_BgZOffset	–
MO_XOffset	–
MO_YOffset	–
MO_XAxis	–
MO_YAxis	–
MO_XSize	–
MO_YSize	–
MO_Angle	–
MO_FgAddSub	–

MO_BgAddSub	-
MO_BurnIn	-
MO_FgRedGain	-
MO_FgGreenGain	-
MO_FgBlueGain	-
MO_FgAlphaGain	-
MO_BgAlphaGain	-

Example

This simple example additively merges the FG over the BG and provides no further options.

```
FuRegisterClass("SimpleMerge", CT_Tool, {
    REGS_Category = "Fuses\\Samples",
    REGS_OpIconString = "SMrg",
    REGS_OpDescription = "Simple Merge",
})

function Create()

    InBackground = self:AddInput("Background", "Background", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })

    InForeground = self:AddInput("Foreground", "Foreground", {
        LINKID_DataType = "Image",
        LINK_Main = 2,
    })

    OutImage = self:AddOutput("Output", "Output", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })

end

function Process(req)
    local bg = InBackground:GetValue(req)
    local fg = InForeground:GetValue(req)

    local out = bg:Copy()
    out = bg:MergeOf(fg, {MO_ApplyMode = "Merge"})

    OutImage:Set(req, out)
end
```

MultiProcessPixels

Summary

The MultiProcessPixels function will process every pixel within a specified range of a source image using the function provided as its last argument. This function is different from ProcessPixels in that the processing will be separated into multiple threads, to take better advantage of multiprocessor systems. The results will be written to a second destination image.

Usage

Image:MultiProcessPixels(*function* threadinitfunc, *table* globalenv, *number* x_start, *number* y_start, *number* x_end, *number* y_end [, *Image* srcimg1 [, ...]], *function* processfunc)

threadinitfunc (*function*, required)

This argument must be provided, but can be either nil. or a function. This can be used to provide a per-thread initialisation function, in case there is a need to calculate/remember something for each thread.

globalenv (*table*, required)

The MultiProcessPixels process function does not have access to the variables in the global environment - it only has access to values passed to it in this table.

x_start, y_start, x_end, y_end (*integer*, required)

These four values are used to specify the range of pixels in the source image which will be affected by the process function. Usually x_start and y_start are set to 0, while x_end and y_end are set to the width and height of the image, respectively.

source_image (*image*, required)

An Image object which will provide the pixels used for the calculations. At least one image is required, but additional images can be specified as well.

process (*function*, required)

A function which will be executed for each pixel in the range specified by earlier arguments. The function will be passed three or more arguments in the form function(x, y, p1, ...) where x and y are the coordinates of the pixel and the remaining arguments are Pixel objects from each of the source images.

Example

The following example implements a very simple Gain using MultiProcessPixels. Note that this is not the recommended way to perform a Gain - see the Image:Gain function for a much faster approach.

```
FuRegisterClass("MultiPxl", CT_Tool, {
    REGS_Category = "Fuses\\Examples",
    REGS_OpIconString = "MltP",
    REGS_OpDescription = "Multi threaded pixel operations Fuse",
})

function Create()
    InGain = self:AddInput("Gain", "Gain", {
        LINKID_DataType = "Number",
        INPID_InputControl = "SliderControl",
        INP_Default = 2.0,
```

```

    })

    InImage1 = self:AddInput("Input 1", "Input1", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })

    InImage2 = self:AddInput("Input 2", "Input2", {
        LINKID_DataType = "Image",
        LINK_Main = 2,
    })

    OutImage = self:AddOutput("Output", "Output", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })
end

-- pixel function
local func = function (x, y, p1, p2)
    p1.R = gain * (p1.R - p2.R)
    p1.G = p1.G - p2.G - bright
    p1.B = var_C * (p1.B - p2.B)
    p1.A = p1.A - p2.A
    return p1
end

function Process(req)
    local img1 = InImage1:GetValue(req)
    local img2 = InImage2:GetValue(req)

    local ingain = InGain:GetValue(req).Value
    local out = Image({IMG_Like = img1})

    -- Must have a valid operation function, and images must be same
    -- dimensions
    if func and (img1.Width == img2.Width) and (img1.Height == img2.
    Height) then

        out:MultiProcessPixels(nil, {gain = ingain , bright=-0.3, var_C=1.5},
        0,0, img1.Width, img1.Height, img1, img2,
        func)

    end

    OutImage:Set(req, out)
end
end

```

OMerge

Summary

OMerge() is a destructive Simple additive merge. It has none of the advanced transformation options found in Image:Merge - allowing only a simple offset of the foreground in x and y in pixels. This can be used to reverse Crop Image

Usage

Image:OMerge(*image* foreground, *number* x_offset, *number* y_offset)

foreground (*image*, required)

An image object to use as the foreground for the merge.

x_offset, y_offset (*number*, optional)

A numeric value which specifies an offset for the Foreground in pixels.

Example

```
function Process(req)
  local bg = InBackground:GetValue(req)
  local fg = InForeground:GetValue(req)

  local out = bg:Copy()
  out:OMerge(fg, 0.75, 0.25)

  OutImage:Set(req, out)
end
```

OXMerge

Summary

OXMerge() is a destructive Simple subtractive merge. It has none of the advanced transformation options found in Image:Merge - allowing only a simple offset of the foreground in x and y in pixels. This can be used to reverse Crop Image

Usage

Image:OXMerge(*image* foreground, *number* x_offset, *number* y_offset)

foreground (*image*, required)

An image object to use as the foreground for the merge.

x_offset, y_offset (*number*, optional)

A numeric value which specifies an offset for the Foreground in pixels.

Example

```
function Process(req)
  local bg = InBackground:GetValue(req)
  local fg = InForeground:GetValue(req)
```

```

local out = bg:Copy()
out:OXMerge(fg, 0.75, 0.25)

OutImage:Set(req, out)
end

```

Resize

Summary

The Resize function resizes an image to the dimensions specified in the functions attributes table. It applies the resized result to the image provided as its first argument.

Usage

Image:Resize(*image* result, table options)

result (*image*, required)

The image object where the results of the resize will be applied. If none is provided, a new image will be created and returned by the function.

attributes (*table*, required)

A table of options which describe the width, height, filter and other parameters used by the resize function. See the Options section below.

Options

Name	Description
RSZ_Filter	A string describing which filter method should be used for the resize. Valid options are : "TopLeft", "Nearest", "Box", "Linear", "BiLinear", "Quadratic", "BiCubic", "Cubic", "BSpline", "CatmulRom", "Gaussian", "Mitchell", "Lanczos", "Sinc", "Bessel"
RSZ_Window	A string that specifies the Windowing method to use with Lanczos and Sinc filter methods. Not required for other filter types. "Hanning", "Hamming", "Blackman", "Kaiser"
RSZ_Width	An integer specifying the width of the result image in pixels
RSZ_Height	An integer specifying the height of the result image in pixels
RSZ_Depth	An integer specifying the color depth of the result image, Usually left nil.
RSZ_XSize	A numeric value representing the horizontal scale of the result image. Provides an alternative to RSZ_Width. A value of 1.0 represents 100%, or no change.
RSZ_YSize	A numeric value representing the vertical scale of the result image. Provides an alternative to RSZ_Height. A value of 1.0 represents 100%, or no change.

Example

```
function Process(req)
    local bg = InBackground:GetValue(req)

    local out = Image({IMG_Like = img, IMG_Width = 1920, IMG_Height = 1080}
    bg:Resize(out, {RSZ_Filter = "Cubic", })

    OutImage:Set(req, out)
end
```

RecycleSAT

Summary

The RecycleSAT function will reduce the reference count for an Images Summed Area Table (SAT) and if the reference count reaches 0, it will release any memory consumed by the table. See UseSAT for more information.

Usage

Image:RecycleSAT()

SamplePixelB

Summary

This function will sample an arbitrary position from the coordinates specified by the first two arguments, and fill the Pixel object p with values from the sampled pixels. The X and Y arguments are floating-point pixel coordinates, with 0,0 being the bottom-left corner and <width-1>,<height-1> being the top-right corner. Unlike GetPixel, if the coordinates do not align exactly with a pixel then bilinear filtering will be performed with neighbouring pixels.

If the coordinates provided are outside the actual bounds of the image, the return value will be a black/transparent pixel. See SamplePixelW and SamplePixelD for functions that treat out of bound sampling in different ways.

Usage

Image:SamplePixelB(*number* x, *number* y, *Pixel* p)

x (*number*, required)

The x coordinate of the pixel to be sampled, where 0 is the left edge and <width-1> is the right edge.

y (*number*, required)

The y coordinate of the pixel to be sampled, where 0 is the bottom edge and <height-1> is the top edge.

p (*pixel*, required)

A Pixel object that will be filled with the results.

SamplePixelD

Summary

This function will sample an arbitrary position from the coordinates specified by the first two arguments, and fill the Pixel object *p* with values from the sampled pixels. The *X* and *Y* arguments are floating-point pixel coordinates, with 0,0 being the bottom-left corner and <width-1>,<height-1> being the top-right corner. Unlike *GetPixel*, if the coordinates do not align exactly with a pixel then bilinear filtering will be performed with neighbouring pixels.

If the coordinates provided are outside the actual bounds of the image, the return value will be the pixel at the edge of the image. For example, if sampling a pixel at coordinates (-30.0, 50.0) the actual pixel sampled would be (0.0, 50.0). See *SamplePixelW* and *SamplePixelB* for functions that treat out of bound sampling in different ways.

Usage

Image:SamplePixelD(*number x*, *number y*, *Pixel p*)

x (number, required)

The *x* coordinate of the pixel to be sampled, where 0 is the left edge and <width-1> is the right edge.

y (number, required)

The *y* coordinate of the pixel to be sampled, where 0 is the bottom edge and <height-1> is the top edge.

p (pixel, required)

A Pixel object that will be filled with the results.

SamplePixelW

Summary

This function will sample an arbitrary position from the coordinates specified by the first two arguments, and fill the Pixel object *p* with values from the sampled pixels. The *X* and *Y* arguments are floating-point pixel coordinates, with 0,0 being the bottom-left corner and <width-1>,<height-1> being the top-right corner. Unlike *GetPixel*, if the coordinates do not align exactly with a pixel then bilinear filtering will be performed with neighbouring pixels.

If the coordinates provided are outside the actual bounds of the image, the coordinates will wrap around the image. For example, if sampling a pixel at coordinates (-30.0, 50.0) the actual pixel sampled would be (width-30.0, 50.0). See *SamplePixelD* and *SamplePixelB* for functions that treat out of bound sampling in different ways.

Usage

Image:SamplePixelD(*number x*, *number y*, *Pixel p*)

x (number, required)

The *x* coordinate of the pixel to be sampled, where 0 is the left edge and <width-1> is the right edge.

y (number, required)

The y coordinate of the pixel to be sampled, where 0 is the bottom edge and <height-1> is the top edge.

p (pixel, required)

A Pixel object that will be filled with the results.

SampleAreaB

Summary

This function will sample an area of the image, using the coordinates specified by the first two arguments, and fill the Pixel object p with the average of the pixels within the area. If the coordinates provided include pixels outside the actual bounds of the image, the pixels will be considered to be black.

See SampleAreaW and SampleAreaD for functions that treat out of bound sampling in different ways.

Before using the SampleArea methods, initial setup must be done, which pre-calculates values needed by the area sampling functions. Be aware however area sampling is a very memory intensive approach and should only be used if "full quality" is required. The setup is done using the UseSAT() function. This maintains a reference count, and so an equivalent RecycleSAT() must be done once you're finished area sampling. Without the RecycleSAT(), Fusion will not be able to free up the pre-calculated values until the whole Image itself is destroyed.

Usage

Image:SampleAreaB(*number* x1, *number* y1, *number* x2, *number* y2, *number* x3, *number* y3, *number* x4, *number* y4, Pixel p)

x1, x2, x3, x4 (number, required)

The x coordinate of the pixel to be sampled.

y1, y2, y3, y4 (number, required)

The y coordinate of the pixel to be sampled.

p (pixel, required)

A Pixel object that will be filled with the results.

SampleAreaD

Summary

This function will sample an arbitrary position from the coordinates specified by the first two arguments, and fill the Pixel object p with values from the sampled pixels. The X and Y arguments are floating-point pixel coordinates, with 0,0 being the bottom-left corner and <width-1>,<height-1> being the top-right corner. Unlike GetPixel, if the coordinates do not align exactly with a pixel then bilinear filtering will be performed with neighbouring pixels.

If the coordinates provided are outside the actual bounds of the image, the return value will be the pixel at the edge of the image. For example, if sampling a pixel at coordinates (-30.0, 50.0) the actual pixel sampled would be (0.0, 50.0).

See SamplePixelW and SamplePixelB for functions that treat out of bound sampling in different ways.

Before using the SampleArea methods, initial setup must be done, which pre-calculates values needed by the area sampling functions. Be aware however area sampling is a very memory intensive approach and should only be used if "full quality" is required. The setup is done using the UseSAT()function. This maintains a reference count, and so an equivalent RecycleSAT() must be done once you're finished area sampling. Without the RecycleSAT(), Fusion will not be able to free up the pre-calculated values until the whole Image itself is destroyed.

Usage

Image:SampleAreaD(*number* x, *number* y, *Pixel* p)

x (number, required)

The x coordinate of the pixel to be sampled, where 0 is the left edge and <width-1> is the right edge.

y (number, required)

The y coordinate of the pixel to be sampled, where 0 is the bottom edge and <height-1> is the top edge.

p (pixel, required)

A Pixel object that will be filled with the results.

SampleAreaW

Summary

This function will sample an area of the image, using the coordinates specified by the first two arguments, and fill the Pixel object p with the average of the pixels within the area. If the coordinates provided include pixels outside the actual bounds of the image, the pixels will be sampled from the opposite side of the image.

See SampleAreaB and SampleAreaD for functions that treat out of bound sampling in different ways.

Before using the SampleArea methods, initial setup must be done, which pre-calculates values needed by the area sampling functions. Be aware however area sampling is a very memory intensive approach and should only be used if "full quality" is required. The setup is done using the UseSAT()function. This maintains a reference count, and so an equivalent RecycleSAT() must be done once you're finished area sampling. Without the RecycleSAT(), Fusion will not be able to free up the pre-calculated values until the whole Image itself is destroyed.

Usage

Image:SampleAreaW(*number* x, *number* y, *Pixel* p)

x1, x2, x3, x4 (number, required)

The x coordinate of the pixel to be sampled.

y1, y2, y3, y4 (number, required)

The y coordinate of the pixel to be sampled.

p (pixel, required)

A Pixel object that will be filled with the results

Saturate

Summary

The Saturate function adjusts the saturation of the image. The result is applied directly to the Image object which calls the function. This function does not return a value.

Usage

Image:Saturate(*number* r, *number* g, *number* b)

number r, g, b (number, required)

The amount by which to adjust the saturation. A value of 1.0 means no change, above 1 is more saturated and less than 1 is desaturated.

Example

A very simple saturate tool.

```
FuRegisterClass("SimpleSaturate", CT_Tool, {
    REGS_Category = "Fuses\\Examples",
    REGS_OpIconString = "SSt",
    REGS_OpDescription = "SimpleSaturate",
})

function Create()
    InSat = self:AddInput("Saturation", "Saturation", {
        LINKID_DataType = "Number",
        INPID_InputControl = "SliderControl",
        INP_Default = 1.0,
    })

    InImage = self:AddInput("Input", "Input", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })

    OutImage = self:AddOutput("Output", "Output", {
        LINKID_DataType = "Image",
        LINK_Main = 1,
    })

end

function Process(req)
    local img      = InImage:GetValue(req)
    local sat      = InSat:GetValue(req).Value

    local newimg   = img:Copy()

    newimg:Saturate(sat, sat, sat) -- This could also desaturate a
    single color
end
```

```
    OutImage:Set(req, newimg)
end
```

SetCanvasColor

Summary

The SetCanvasColor function is used to set the canvas color values of an Image.

The canvas color is the "default" pixel color, and is used for any part of the image which is not explicitly defined by pixels. This is usually black/transparent, but can be different after certain operations, such as inverting the image.

Usage

SetCanvasColor(*object* pixel)

pixel

The Pixel object that is used to set the color values of the image's canvas.

Example

```
local p = Pixel()

p:Clear()
img:SetCanvasColor(p)

print("Image canvas is now black/transparent")
```

SetPixel

Summary

The SetPixel function is used to set the value of a specific pixel in an Image. This uses actual pixel coordinates, and must always be within image bounds.

Usage

Image:SetPixel(*integer* x_position, *integer* y_position, *object* pixel)

x_position

The position of the pixel to set on the x axis

y_position

The position of the pixel to set on the y axis

pixel

The pixel object to be assigned to the image.

Example

The following example is taken from the SourceTest.Fuse example found at [Example_Fuses](#)

```
local img = Image(imgattrs)

local random = math.random -- faster in a local
```

```

local p = Pixel({A=1})

for y=0,Height-1 do
    if self.Status ~= "OK" then break end

    for x=0,Width-1 do
        p.R = random()
        p.G = random()
        p.B = random()
        img:SetPixel(x,y, p)
    end
end

OutImage:Set(req, img)

```

Transform

Summary

The Transform method can be used to change the scale, angle and position of an image. The results of the transform will be copied into the image provided as the first argument. If the first argument is set to nil, the method will return a new Image object containing the results.

The defaults for XF_XOffset and XF_YOffset are zero, so if you don't want to do translation you still need to specify 0.5 for each one if you don't want your image to end up in the bottom left corner.

Usage

image = Image:Transform(*image* dest_image, *table* taglist)

dest_image (*image*, required)

The image to write the transform results into. May be nil, in which case an image is created.

taglist (*table*, required)

The taglist argument is a table containing entries which describe the image transformation.

Name	Description
XF_Angle	The angle of the transformed image in degrees.
XF_XAxis	The X coordinate for the transformations axis (or pivot).
XF_YAxis	The Y coordinate for the transformations axis (or pivot).
XF_EdgeMode	A string which describes which technique to use to handle edges of the image. Valid options are "Black", "Canvas", "Wrap", or "Duplicate".
XF_XOffset	The x coordinate for the transformations center.
XF_YOffset	The y coordinate for the transformations center.
XF_XSize	The scale of the transformed image along the x axis.
XF_YSize	The scale of the transformed image along the y axis.

Return

The results of the transformation are returned as an Image object, or nil if the operation failed. If dest_image was provided, that will be returned.

Example

The following example is the process() event from the Fuse.

```
edge_modes = {"Black", "Canvas", "Wrap", "Duplicate"}

function Process(req)
    local img      = InImage:GetValue(req)
    local center   = InCenter:GetValue(req)
    local pivot    = InPivot:GetValue(req)
    local sizex    = InSizeX:GetValue(req).Value
    local sizey    = InSizeY:GetValue(req).Value
    local angle    = InAngle:GetValue(req).Value
    local edges    = InEdges:GetValue(req).Value

    if locked then
        sizey = sizex
    end

    out = img:Transform(nil, {
        XF_XOffset = center.X,
        XF_YOffset = center.Y,
        XF_XAxis   = pivot.X,
        XF_YAxis   = pivot.Y,
        XF_XSize   = sizex,
        XF_YSize   = sizey,
        XF_Angle   = angle,
        XF_EdgeMode = edge_modes[edges+1],
    })

    OutImage:Set(req, out)
end
```

UseSAT

Summary

The UseSAT function must be called before using the SampleArea functions; SampleAreaB, SampleAreaD, and SampleAreaW. It will create a Summed Area Table (SAT), containing pre-calculated values for the entire image. If the SAT has already been created for the Image object this function will increase the reference count for the SAT. This function does not return a value - the SAT is attached directly to the Image object.

When the SAT is no longer required, use the RecycleSAT function to release the memory consumed by the table.

Usage

Image:UseSAT()

Request

Methods

IsStampOnly	Returns a boolean which indicates whether the current request is in Proxy mode.
IsQuick	Returns a boolean which indicates whether the current request is in HiQ mode.
IsNoMotionBlur	Returns a boolean which indicates whether the current request should include motion blur.
GetTime	Returns the current frame of the request.

Members

Time	The current frame of the request.
BaseTime	Returns a boolean which indicates whether the current request is in HiQ mode.

Tips for Request

IsPreCalc():	Returns true if the current request is a precalc request.
GetFlags():	Get all the request's flags (IsStampOnly, IsQuick, ...) as bits of an integer

Domain of Definition

To use DoD in a fuse, you will need these methods

Name	Description
GetInputDoD(inp):	Returns the specified input's DoD. You mustn't access pixel coordinates outside of this when working on the input image data. The canvas color value should be substituted instead, something that GetPixel() won't do for you automatically.
GetRoI():	Returns the requested region of interest. You may return an image with a DoD that is larger or smaller, but if you want to gain as much speed as possible, don't waste time calculating pixels outside of this. During precalc requests, this is nil. For regular process requests, this should end up as the ValidWindow attribute of your output image.

Both functions return ImageDomain objects. They are similar to FuRectInt in that they store information about the area that contains valid pixel data. You can access these coordinates using .left, .bottom, .right and .top as if it was a FuRectInt.

Pixel

Methods

Clear	Zeroes all pixel values to black.
-------	-----------------------------------

Members

R	Red
G	Green
B	Blue
A	Alpha
Z	Depth
U	U texture coordinate
V	V texture coordinate
W	W texture coordinate
Coverage	Fraction of pixel covered by foreground object
ObjectID	Unique integer identifier for the pixel's object
MaterialID	Unique integer identifier for the pixel's material
NX	X component of pixel's surface normal vector
NY	Y component of pixel's surface normal vector
NZ	Z component of pixel's surface normal vector
BgR	Red component of background pixel fragment
BgG	Green component of background pixel fragment
BgB	Blue component of background pixel fragment
BgA	Alpha component of background pixel fragment
VectorX	X component of pixel's motion vector
VectorY	Y component of pixel's motion vector
BackVectorX	X component of pixel's reverse motion vector
BackVectorY	Y component of pixel's reverse motion vector
DisparityX	X component of pixel's Stereo Disparity difference
DisparityY	Y component of pixel's Stereo Disparity difference
PositionX	X position of pixel in 3D world space
PositionY	Y position of pixel in 3D world space
PositionZ	Z position of pixel in 3D world space

Other

No attributes.

ColorMatrixFull

Summary

The ColorMatrixFull function is used to create a Matrix object with four elements, used to manipulate the Red, Green, Blue and Alpha channels. For a three element Matrix, see ColorMatrix instead.

For a description of what a ColorMatrix is, and where it might be useful, see : Using the ColorMatrix.

Usage

ColorMatrixFull()

Example

The following example uses a ColorMatrixFull object to perform a contrast operation on an image.

Using the ColorMatrix

Introduction

In Using the Matrix we saw how a Matrix object could be used to collect multiple spatial transformations on an image. In this article we explore how a variant on that technique can also be used to perform color operations.

The key to this approach is that you consider the values in the red green and blue channels as coordinates in x, y and z instead. Once you do that, it becomes apparent that operations like Gain are really no different than spatial transformations like Scale.

In fact, Gain is identical to scaling the image, while brightness is nothing more than a translation of the image. A contrast operation is nothing more than a scale centered around 0.5 instead of 0.0. Even the conversion to YUV can be represented in this way. This allows us to build up several 'linear' color operations into one operation, and then apply them as a single pass. We can even use a matrix to swap channels, or mix them together.

Non linear color operations (like gamma) cannot be represented this way in a Matrix.

Fusion provides a ColorMatrix object for RGB image operations, and the ColorMatrixFull object for RGBA images.

The Math of the Matrix

The 'ColorMatrix' is a 4x4 matrix, so you can use it to affect RGB. The 'ColorMatrixFull' is a 5x5 matrix, which adds Alpha to the RGB color channels.

In the terminology of Matrix mathematics, an "identity matrix" is one that doesn't change anything. A 4x4 identity matrix would look like:

	r	g	b	
R	1	0	0	0
G	0	1	0	0
B	0	0	1	0
	0	0	0	1

We could say that uppercase R, G and B are going to be the results, and lowercase r, g and b are the source/input values. You can completely ignore the bottom row, but not the right column.

So just looking at the first row, it says the R result will contain 1 of r, 0 of g and 0 of b (or $1 * r + 0 * g + 0 * b$). And the second row says the R result will contain 0 of r, 1 of g and 0 of b. And similar for the 3rd row.

So far we just ignored the right column, but you can consider the source/input value for the right column to be 1.0. So that would make the first row of the matrix produce results for $R = 1 * r + 0 * g + 0 * b + 0 * 1.0$.

With that, we can do "brightness" by increasing or decreasing how much of the 1.0 input value is included in the result. Let's say we wanted a brightness of 0.5 in the R result. That would make the R row of the matrix:

	r	g	b	1.0
R	1	0	0	0.5
R	$= 1 * r + 0 * g + 0 * b + 0.5 * 1.0$			
	$= 1 * r + 0.5 * 1.0$			
	$= r + 0.5$			

If instead we wanted a "gain" of 2.0 (so we want the R result to have 2 times r), that would make the R row of the matrix:

	r	g	b	1.0
R	2	0	0	0
R	$= 2 * r + 0 * g + 0 * b + 0 * 1.0$			
	$= 2 * r$			

Say we wanted to put g into the resulting R, (copying the green channel into the Red channel of the output) then the first row of the matrix would be:

	r	g	b	1.0
R	0	1	0	0
B	$= 0 * r + 1 * g + 0 * b + 0 * 1.0$			
	$= 1 * g$			
	$= g$			

If you wanted R to be the "luminance" of rgb, the first row of the matrix would be:

	r	g	b	1.0
R	0.299	0.587	0.114	0
R	$= 0.299 * r + 0.587 * g + 0.114 * b + 0.0 * 1.0$			
	$= 0.299 * r + 0.587 * g + 0.114 * b$			

If we want R to be an inverted version of r, we want to do "1.0 - r", which means we want to make r negative (gain of -1.0) and then add 1.0 (brightness of 1.0), so the first row of the matrix would be:

	r	g	b	1.0
R	-1	0	0	1
R	= -1 * r + 0 * g + 0 * b + 1 * 1.0			
	= -1 * r + 1 * 1.0			
	= -r + 1			
	= 1 - r			

Using Methods

The `ColorMatrix` and `ColorMatrixFull` objects expose several methods that can make common operations much simpler. For example, we can use the `Scale` function to simplify the Gain example in the section above.

```
local img = InImg:GetValue(req)
m = ColorMatrix()
m:Scale(0.5, 0.5, 0.5)
```

The advantage to this approach is that the `Scale` method takes care of preserving the existing transformations applied to the Matrix. Our original example would have overwritten any existing transformations.

Using a similar technique, the `Offset` method can be used to perform a brightness operation.

```
local m = ColorMatrixFull()
m:Offset(r, g, b, a)
```

You can combine operations by simply applying them in turn.

```
local m = ColorMatrixFull()
m:Offset(-0.5, -0.5, -0.5, 0)
m:Scale(1, 0.5, 0.25, 1)
m:Offset(0.5, 0.5, 0.5, 0)
```

You can also combine the operations in separate matrices by multiplying them together.

```
local m1 = ColorMatrixFull()
m1:Offset(-0.5, -0.5, -0.5, 0)
local m2 = ColorMatrixFull()
m2:Scale(1, 0.5, 0.25, 1)
local m = m1 * m2
```

You can find a complete list of the methods available at the `ColorMatrix` and `ColorMatrixFull` object reference pages.

Editing the Matrix

Sometimes it is necessary to manipulate the matrix directly. Each matrix object exposes individual elements as properties. To access the first element in the first row, we would use the property `matrix.n11`, the second element would be `matrix.n21`, then `matrix.n31` and so on. This is best demonstrated by the following code, which would print a table of all the elements in a 4x4 `ColorMatrix`, organized as in our examples above.

```
m = ColorMatrix()
print("", "r", "g", "b", "1")
print("R", m.n11, m.n21, m.n31, m.n41)
print("G", m.n12, m.n22, m.n32, m.n42)
print("B", m.n13, m.n23, m.n33, m.n43)
print("A", m.n14, m.n24, m.n34, m.n44)
```

The following function could be used to copy a `ColorMatrix`.

```
function CopyColorMatrix(m)
    local new_m = ColorMatrix()
    new_m.n11 = m.n11
    new_m.n21 = m.n21
    new_m.n31 = m.n31
    new_m.n41 = m.n41
    new_m.n12 = m.n12
    new_m.n22 = m.n22
    new_m.n32 = m.n32
    new_m.n42 = m.n42
    new_m.n13 = m.n13
    new_m.n23 = m.n23
    new_m.n33 = m.n33
    new_m.n43 = m.n43
    return new_m
end
```

The following could be used to apply a gain of 0.5 to each pixel in an image.

```
local img = InImg:GetValue(req)
m = ColorMatrix()
m.n41 = 0.5
m.n42 = 0.5
m.n43 = 0.5
```

Applying the matrix

Once the matrix has been created, you can apply it to the image using the Image objects `ApplyMatrix` and `ApplyMatrixOf` functions.